What This Project Is About

Problem Setting

1. There is a vertical corridor with n rooms and n one-way gates (you can only move upward).
2. Room j holds a keycard with value x_j.
3. Gate j requires a clearance of at least y_j to open.
4. You start at some room i, pick up its card, and climb upward.
5. Every room you pass adds its card to your collection, so your effective card when facing gate t is $\max(x_t, x_{t-1}, \ldots, x_i)$.
6. Gates keep opening as long as your effective card meets their threshold; the moment one gate fails, your journey stops.

What You Are Asked To Do

1. Write the solve function (in solution.cpp) that returns an array: for every starting room i, how many gates can be opened if you start there.
2. The complexity must be O(n): an $O(n^2)$ brute force is not accepted.
3. Write a PDF report containing:

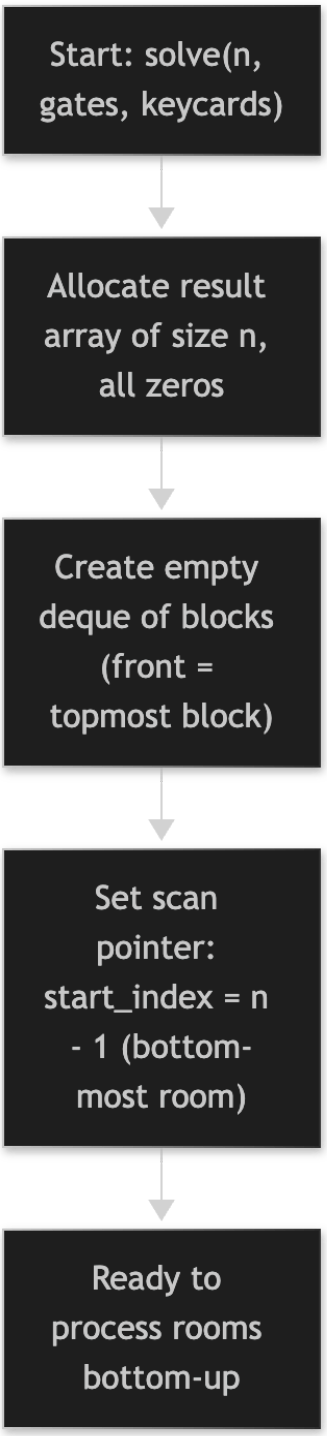
- A description of the data structure (monotonic stack: a stack whose elements are kept in monotonic order; violating elements are popped during push/query).
- Why a stack is chosen over a queue (a queue requires removal from both ends, loses the LIFO property, and makes range queries less efficient).
- A complexity analysis proving O(n) via amortization: each block is pushed at most once and popped at most once.

The Core Technique

Process rooms from bottom to top. Keep segment blocks in a stack/deque, each with three values: need (the block's lowest gate), gain (the maximum keycard inside the block), and cnt (the number of gates). When a new room is processed, pop every block its card can penetrate, merge them into one new

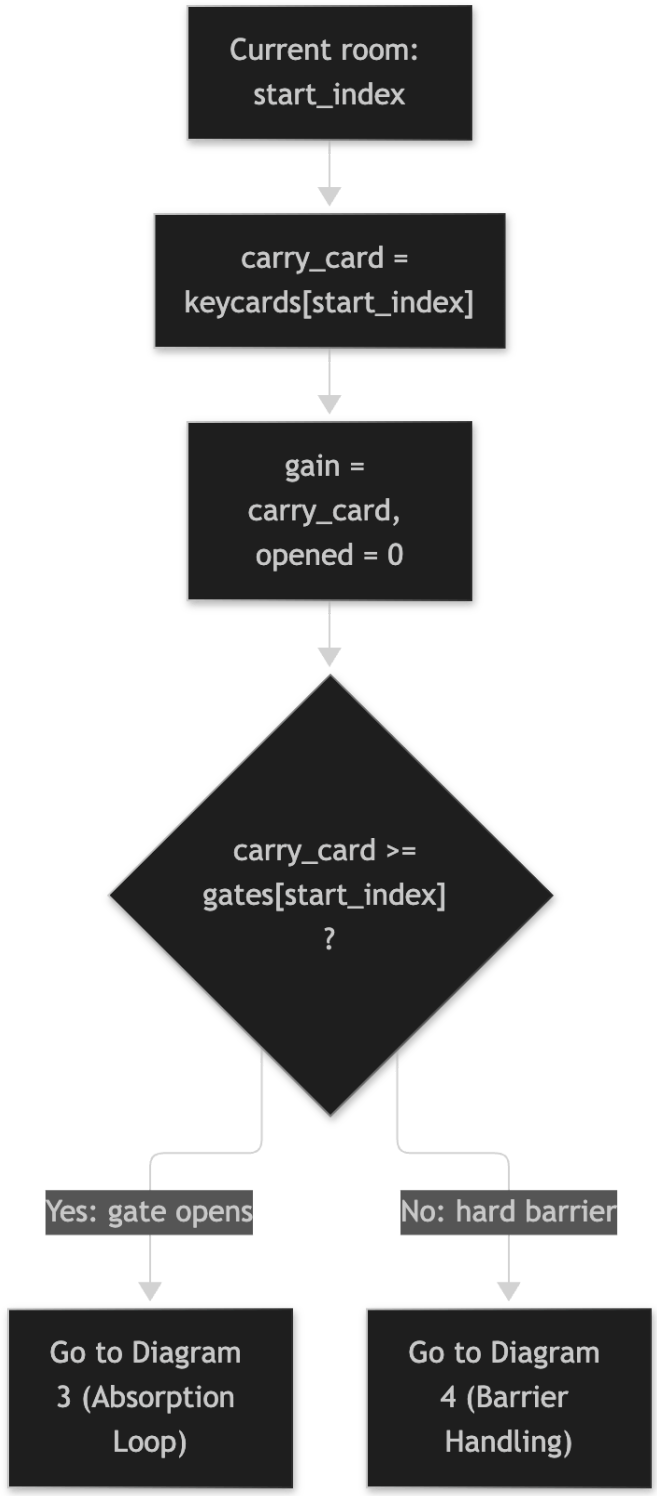
block, and record the answer. That is the monotonic stack pattern that keeps the total work linear.

Diagram 1: Initialization Phase



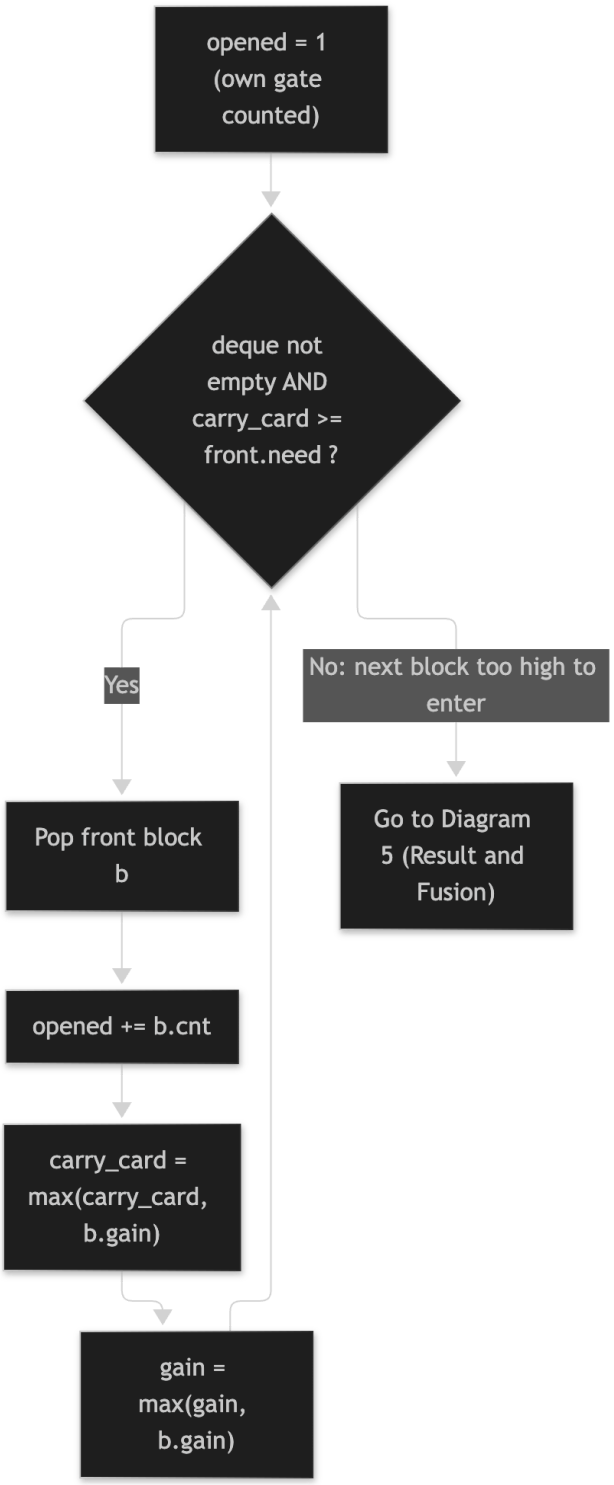
Purpose: prepare the output storage, the block container, and the scan direction. The algorithm processes rooms from the bottom of the corridor upward, so the pointer starts at the last index.

Diagram 2: Card Pickup and Own-Gate Check



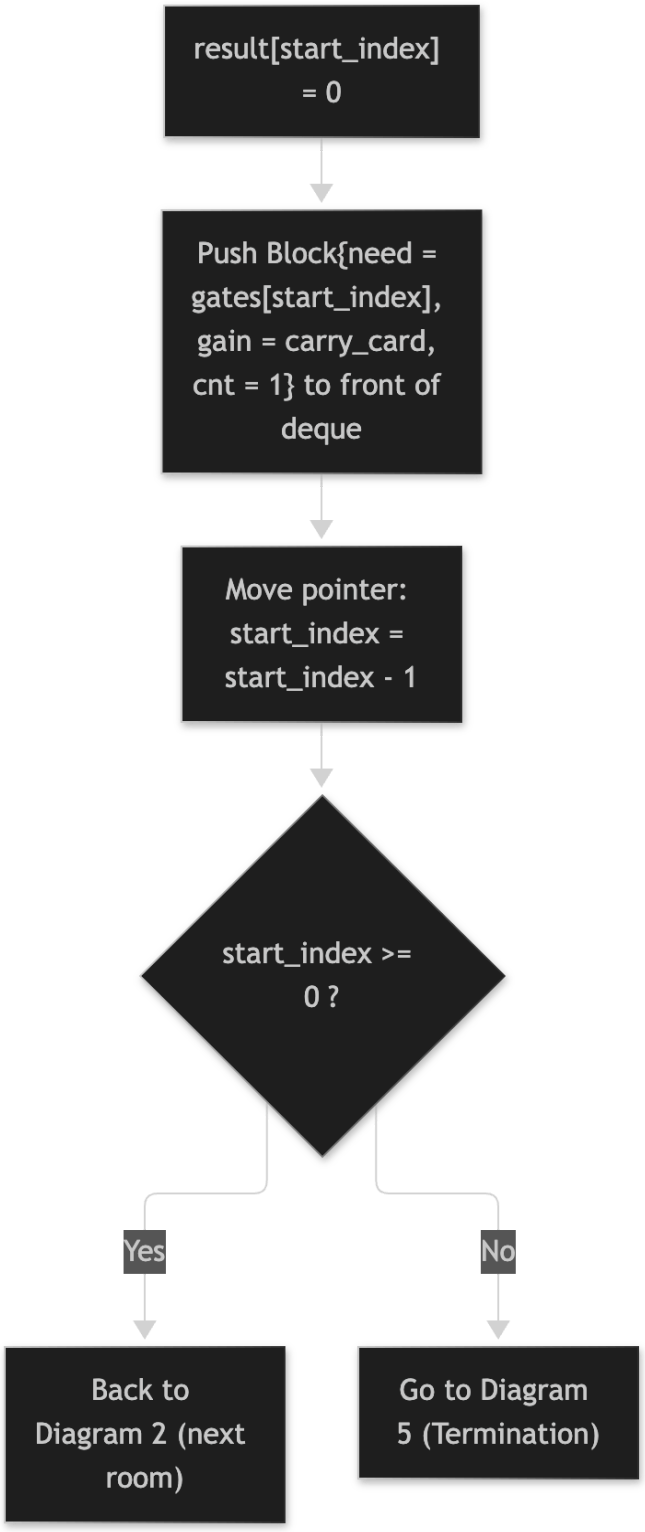
Purpose: this is the single decision point of the algorithm. The traveler's own keycard is tested against the gate of the room they start in. Success leads into the merge loop; failure marks the room as an impassable barrier.

Diagram 3: Absorption Loop (Monotonic Stack Pop Phase)



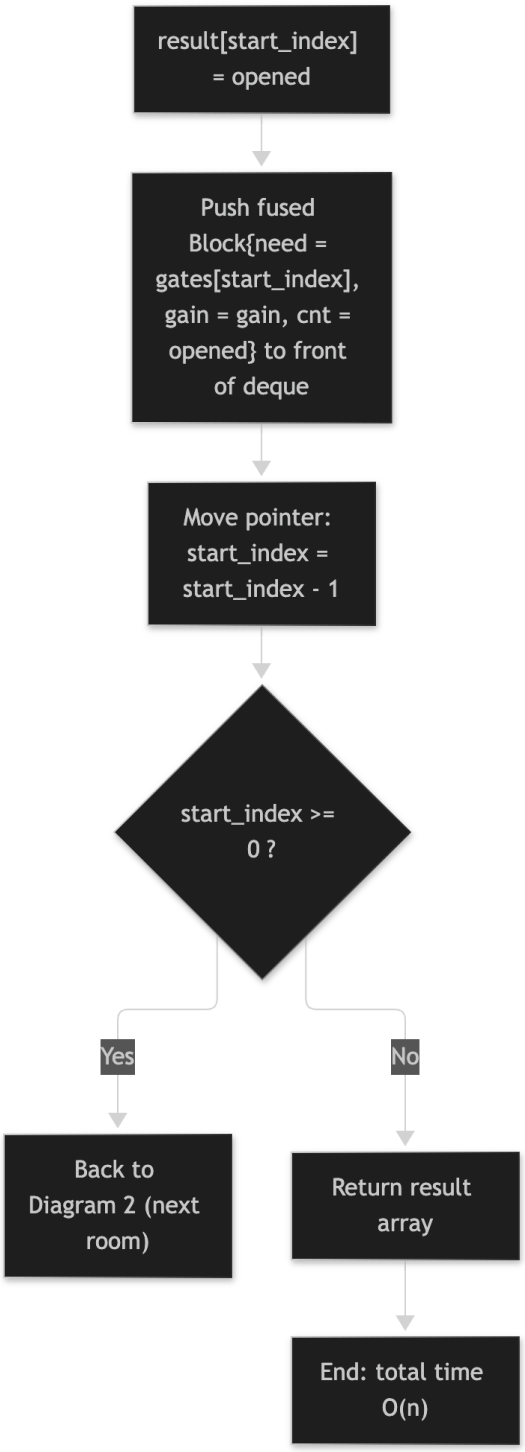
Purpose: this loop absorbs every pre-existing block whose entry threshold the current card can satisfy. Each popped block contributes its gate count and its maximum keycard, which may unlock even higher blocks in the next iteration. Because each block is popped at most once across the entire run, this loop is amortized O(1) per room.

Diagram 4: Barrier Handling (Failed Gate)



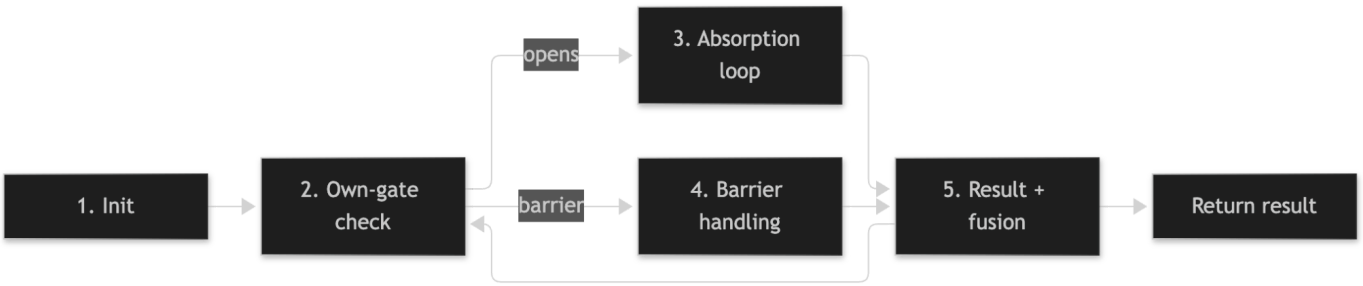
Purpose: a room whose own gate cannot be opened contributes nothing to the answer, but it must still be recorded as a one-gate block. Any future traveler starting from below must meet this block's need threshold to get past it.

Diagram 5: Result Recording, Fusion, and Termination



Purpose: after absorption finishes, the entire traversed chain is collapsed into a single fused block. Its need is the start gate's threshold (the lowest gate in the chain), its gain is the maximum keycard found anywhere in the chain, and its cnt is the total number of gates opened. The pointer then moves to the next room above; when no rooms remain, the result array is complete.

How the Pieces Fit Together



Overall cycle: every room passes through Diagram 2 exactly once, then flows into either Diagram 3 or Diagram 4, and both paths converge in Diagram 5 before looping back to the next room. The only repeated inner structure is the pop loop in Diagram 3, which is what keeps the total work linear at O(n).